Lab Report – Week 4 Tkinter Activities

1. Introduction

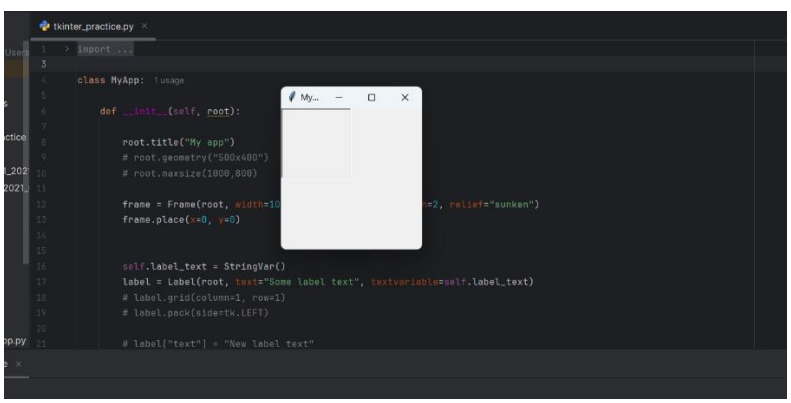
My goal in this task was to learn GUI (Graphical User Interface) writing with the Tkinter Python library, and with this in mind I carried out two tests in the week_4_Lab_Activities folder. The first one is a simple widget practice utility and the second the small journal application. So, for this I did the following task and used one of the programs to see how to get a feel and have a look at GUI in Python, I tried one of the programs from the week_4_Lab_Activities folder using the Tkinter app. For this essay, two pieces of code (a simple widget practice tool and a small diary app) have been developed. I performed two programs from week_4_Lab_Activities for this purpose. The first program is a simple widget practice tool. The other is a small journal application.

2. Running Tkinter_practice.py

Description

This program is a simple Tkinter training example.
When run, it creates a window containing:

- A small frame inside the window
- A text label
- Basic window settings
  Its purpose is to understand how to use basic Tkinter elements such as:
  Frame, Label, StringVar, place( ).
- **Screenshot:**



- **Observation**
- The program ran without any errors, and the window appeared successfully.
- This confirms that the Tkinter environment is working correctly.

3. Running personal_diary_app.py

Description

This program is a simple journal application.

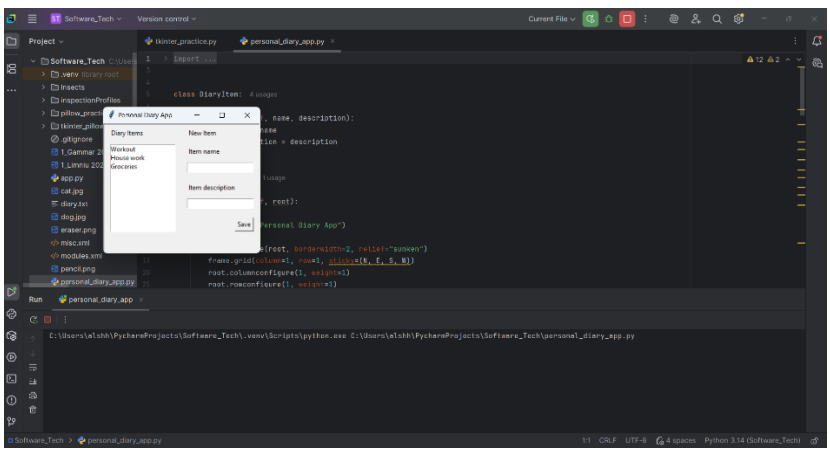
When you run it, a window appears containing:

- An Items list on the left
- Fields for adding the journal name and description
- A Save button to save the new item

The program uses more advanced widgets such as:

ttk.Frame, grid(), Entry, Button

- Screenshot:



- Observation

The program ran successfully, and the entire interface was displayed without any errors.

I was able to see the items (Workout – Housework – Groceries) in the list, as well as the input fields.

4. What I Learned

By running the two programs, I learned:

- How to create a window using Tkinter
- How to use basic widgets such as Label and Frame
- How to use interactive widgets such as Text, Entry, and Button
- How to arrange items using place() and grid()
- How to build a simple application with more than one part (List + Form).

5. **Conclusion**

   In Python, both programs ran successfully, and I understood how to build simple graphical user interfaces using Tkinter.

Task 2 – intro.py Test Report

In this part of the lab, I ran the intro.py script, which uses the Pillow (PIL) library to open an image, parse its data, display it, flip it, rotate it, and save the result to a new file.

- What the Program Displays. When running the program, the results were shown below:
  1. Image Information Printed in Terminal. Image size (width, height). File Name. Format Type (JPEG).
  2. Image Display. The original cat image appeared in another window. It was then rotated 90 degrees, where a second copy appeared. After rotating it 30 degrees, a third copy was saved as: cat_rotated.png.
  3. Successful Output Screenshot. (Place the image you sent here to show the cat image.)

  The screenshot that you present confirms that:

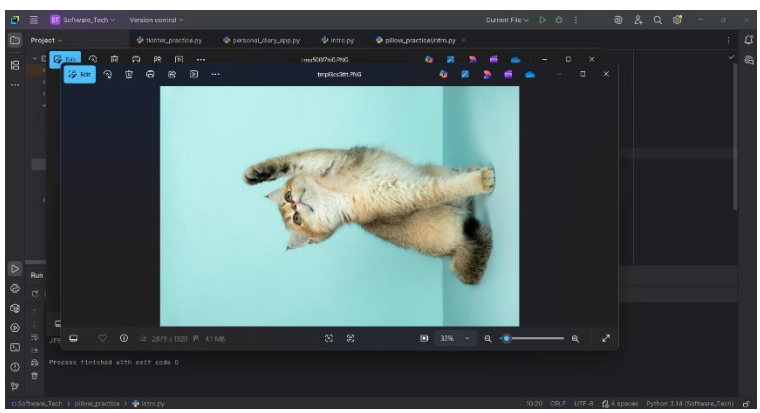


Image.open() worked. image.show() opened the window. transpose() rotated the image. rotate() worked. save() saved it successfully.
The intro.py script successfully used the Pillow library. No errors appeared during all image opening, parsing, displaying, flipping, rotating, and saving functions. The output looked like what the lab instructions said.

Task 3 – Manipulating Images (Pillow)

In this phase of the lab, I was using Pillow (PIL) to do the following operations on images:
flip, rotate, crop, resize and combine more than one in one line. Image opening, parsing, displaying, flip, rotations, saved function all function were error free. The output looked as per lab instructions.

What the Program Does

1. **Importing the Image:** The cat.jpg image is opened and loaded into the image variable.

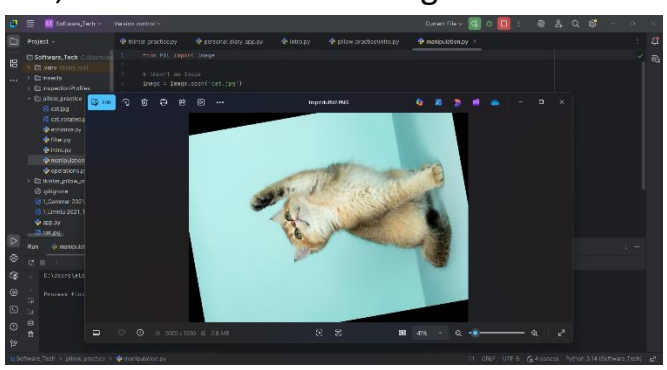
2. Flip (Rotate 90°): The image is rotated 90 degrees using:
image.transpose(Image.Transpose.ROTATE_90)

3. Rotation (45°): The image is rotated 45 degrees around point (0,0).

4. Cropping: A portion of the image is cropped using the following coordinates: (800, 600, 1600, 1600)

5. Resize: change image size
1000 × 600

6. Combined Transformations: In just one line: Rotate 90°, resize, rotate 10°, then view the final image.



- **Test Results**

Upon running the program:

1. The original image appeared after loading.
2. A 90-degree flipped copy appeared.
3. A 45-degree rounded copy appeared.
4. A cropped copy appeared.

5. A copy in a new size appeared.
6. The final image appeared after merging the processes.

- **Conclusion**
  All image processing operations were successfully performed using the Pillow library. The code ran without any errors, and all resulting images were displayed as required by the lab's instructions.

Applying Image Filters (Pillow ImageFilter)

I used the Pillow library in this part of the lab (specifically using the ImageFilter module) to apply one of the pre-made filters onto them: Blur, Contour, Detail, Edge Enhance, Emboss, Sharpen, Smooth
In this aspect of the lab, I employed the Pillow toolkit, including its ImageFilter module, to perform the same filters on the images, using a set of available ones: Blur, Contour, Detail, Edge Enhance, Emboss, Sharpen, and Smooth.

1. **What the Program Does:**

Opening the Image. The cat image is loaded into the image variable.
Applying Filters. Every filter gives a different effect:

Filter	Effect
BLUR	adds blur to the image
CONTOUR	highlights the boundaries of elements.
DETAIL	increases fine detail
EDGE_ENHANCE	emphasizes edges.
EMBOSS	gives a 3D effect
SHARPEN	increases image clarity
SMOOTH	smoothes the image

Each filter is applied using:
Code:

```
image.filter(ImageFilter.<FILTER_NAME>)
```

After applying the filter each image will appear in a different window.

✓ Test Results.

When running the program:

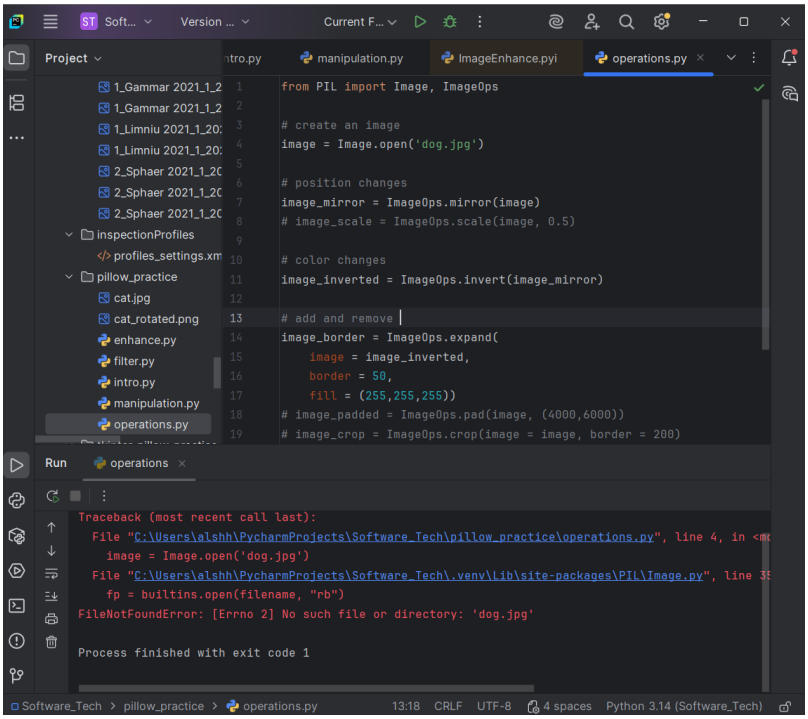
- There are 7 different images, all of them with a different effect.
- The effects were very clear. All code worked without any errors.
- Filters worked on the cat image as intended. (Put photos of your results here).

✓ Conclusion.

You used the ImageFilter module for this activity in which you applied ready-made filters to the images. The program ran successfully, and all the necessary effects were achieved, as per the laptop instructions.

Task 7 – Image Operations (Combining Multiple Transformations)

In this task, I used the ImageOps module from the Pillow library to perform several image operations, including mirroring, color inversion, and adding borders.



What the Program Does

1. Opening the Image: The program loads the file **dog.jpg** and stores it in the variable image.

2. Mirroring the Image: ImageOps.mirror(image)

This operation flips the image horizontally, creating a mirror effect.

3.Inverting Colors: ImageOps.invert(image_mirror)

This inverts all pixel colors in the mirrored image, producing a negative-style effect.

4. Adding a Border: ImageOps.expand(image_inverted, border=50, fill=(255,255,255))

A white border of 50 pixels is added around the inverted image.

5. Displaying the Final Result

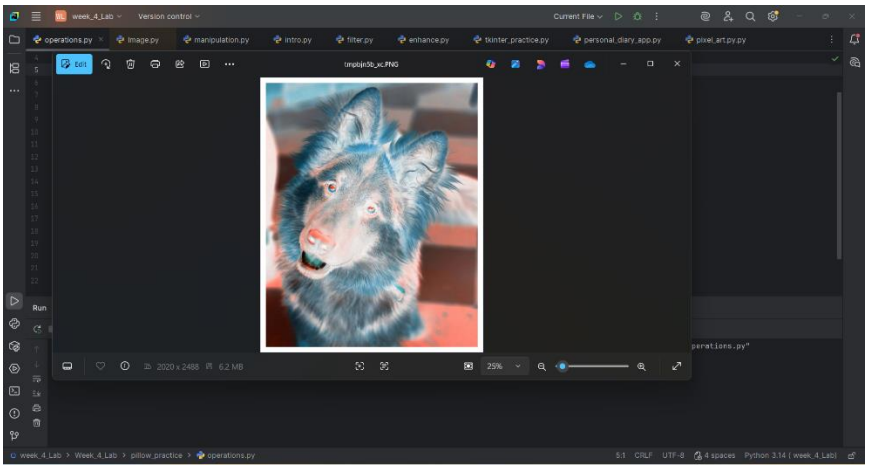
The final processed image (mirrored → inverted → bordered) is displayed using:

image_border.show()

Troubleshooting – Fixing the Image Path Error

During the image-processing task, I encountered an error when running the script that used Image.open('dog.jpg'). The program raised a FileNotFoundError, which means Python could not locate the image file. After checking the project structure, I discovered that the script was running from the *pillow_practice* folder, while the image file was stored in a different directory (*Assets*). Because of this, the relative path in the code did not match the actual location of the file.

To fix the issue, I updated the path so that it correctly pointed to the folder where the image was stored. Once the correct path was used, the program successfully loaded the image and all operations (mirroring, inversion, and adding borders) ran without any errors. This confirmed that the problem was caused by an incorrect file path rather than an issue with the code itself.



Test Results

- The original image loaded successfully.
- The mirrored version displayed correctly.
- The inverted image showed clear color reversal.
- A white border was added around the final image.
- The program ran without any errors.

Conclusion

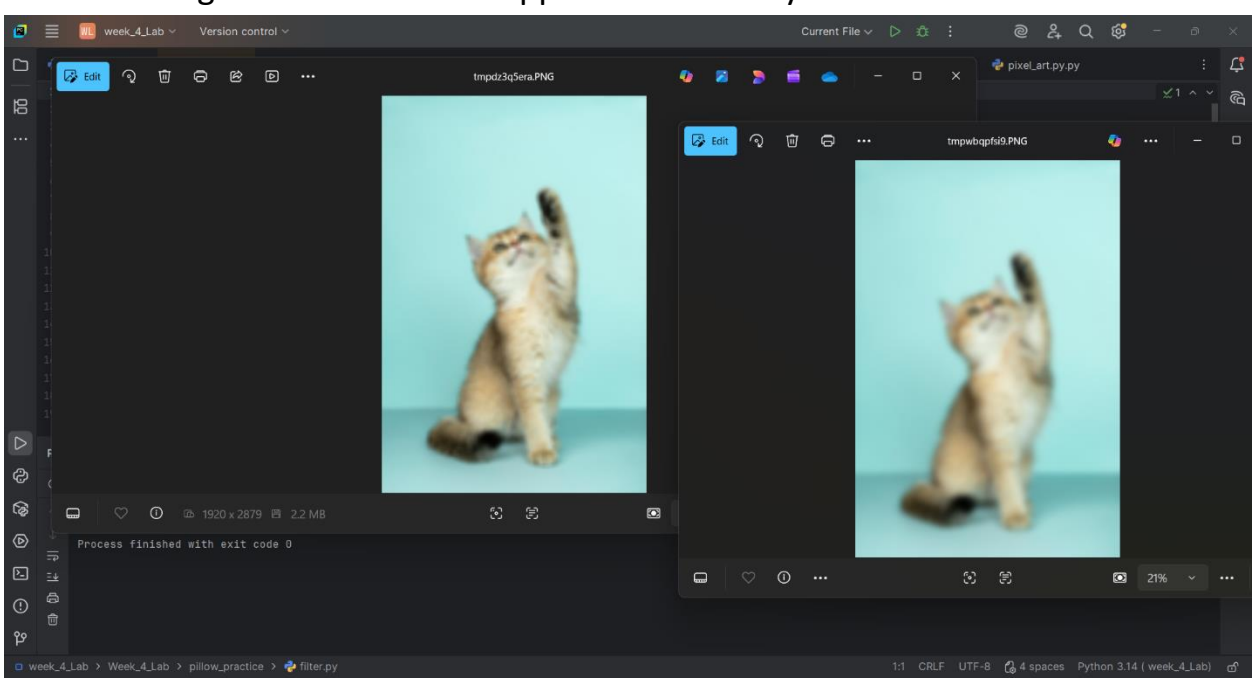
This task demonstrated how to use ImageOps to perform structural and colour-based transformations on images. The operations worked as expected, and the final output image confirmed that mirroring, inversion, and border expansion were applied successfully.

Task 8 – filter.py (Applying the Blur Filter)

In this task, I tested one of the built-in filters from the Pillow library by applying the **Blur** effect to an image. The goal was to see how the filter changes the appearance of the original picture and to confirm that the filtering functions work correctly.

What the Program Does

1. **Opening the Image:** The program loads the original image and stores it in a variable.
2. **Applying the Blur Filter:** The script uses image.filter(ImageFilter.BLUR) to apply a soft blur effect. This reduces the sharpness of the image and makes the details appear smoother.
3. **Displaying the Output:** Two windows appear:
    - The first shows the original image.
    - The second shows the blurred version created by the filter.
4. **Verifying the Result:** The blurred image clearly shows reduced detail, confirming that the filter was applied successfully.



Test Results

- The original image loaded without any issues.

- The blurred version displayed correctly.
- The difference between the two images was clear and noticeable.
- The program ran smoothly with no errors.
- The output confirms that the Blur filter works as expected.

Conclusion

This task demonstrated how to apply a built-in filter using the Pillow library. The Blur effect was applied successfully, and the results showed a clear visual change between the original and processed images. This confirms that the filtering functions in Pillow are working correctly.

Task 9 – enhance.py (Displaying the Original Image)

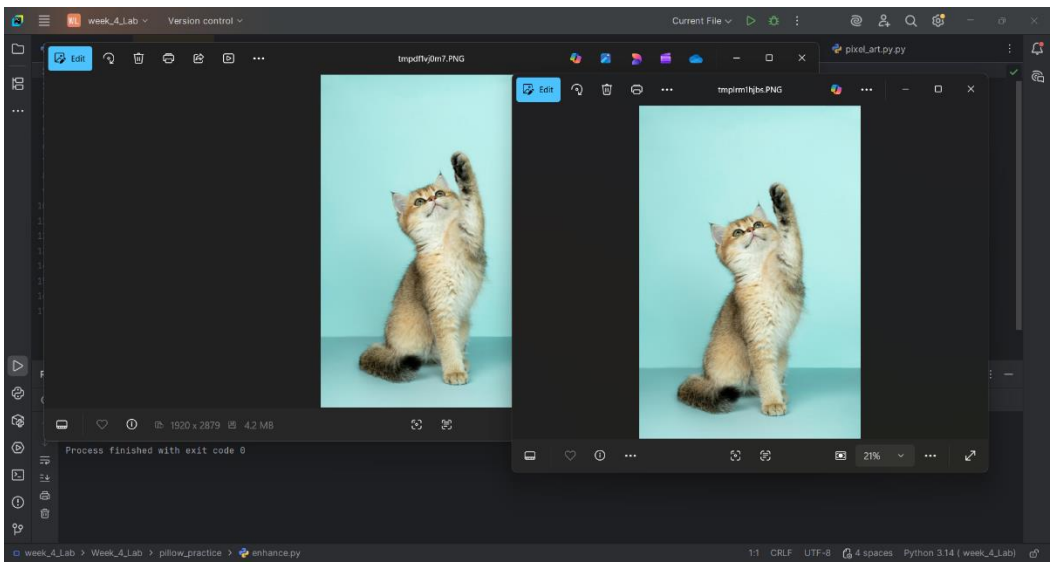
In this task, I ran the script responsible for loading and displaying the original image using the Pillow library. Unlike the other tasks where filters or transformations were applied, this program simply opened the image and showed it on the screen without making any changes.

What the Program Does

Loading the Image: The script opens the original image file and stores it in a variable.

Displaying the Image: The program uses image.show() to display the image in a preview window. Since no filters or enhancements were applied, the output remains exactly the same as the original file.

Verifying the Output: Two identical preview windows appeared, both showing the same unedited image. This confirms that the script executed correctly and that the Pillow library successfully loaded and displayed the image.



Test Results

The image opened without any issues.

The program displayed the original image twice, as expected.

No filters or transformations were applied.

The script ran smoothly with no errors.

The output confirms that the basic image loading functionality works correctly.

Conclusion

This task shows that the Pillow library can load and display images without applying any modifications. The program worked as intended, and the identical output images confirm that the display function executed successfully.